DEVELOPER BRIEFING · FOR THE SAPIENT (CHIRONBUILDS) MAINTAINER

The x402 fork: what changed in Sapient

A change inventory for the chironbuilds/tari-wallet fork branch x402/quote-and-signonly — new request kinds, the sign-only build path, an SDK bug found on the way, and live-testnet verification results.

PotR3288 · October 31, 2026 · base chironbuilds/tari-wallet @ e2f7bf9 (0.6.17)

Branch x402/quote-and-signonly at aee37c8 · 278 ext + 34 pkg + 24 facilitator tests green

Companion explainer for a general audience: x402-over-tari-ootle.pdf

1. The gap this fills

Sapient in 0.6.17 already had the hard parts: htlcFund, htlcClaim and htlcRefund wired through the RPC surface, stealth transfers, and scriptPathSpend reported honestly in tari_getCapabilities. What it did not have was any way to **produce a signed transaction without broadcasting it** — every path ended in submit. That single gap is what blocks settlement flows where a third party (the Ootle indexer, acting as an x402 facilitator) must hold a signed payload and broadcast it later. The fork adds that mode plus the payment-terms plumbing around it, with a deliberately small diff surface: two new request kinds, one new background module, popup rendering, and docs.

2. Change inventory

File / area	Change	Notes
src/lib/messages.ts	New kinds x402Pay and x402SignClaim; request status gains signed-unbroadcast; WalletCapabilities.signOnly field doc.	Types mirror the facilitator spec; the status is distinct from approved/submitted so popup history never implies funds moved.
src/background/index.ts	Dispatch cases for both kinds; a validation gate before the execution switch; capabilities report signOnly: isLocal.	Validation: hashLockHex must be 64 lowercase hex (this also closes an audited htlcFund weakness), refundEpoch bounded, expiresAt in the future, amount > 0, claimantAddress well-formed — each rejection is its own actionable error.
src/background/x402SignOnly.ts (new)	Build/sign/seal-without-submit: resolve the funded HTLC output, buildScriptPathWitness with the preimage + claim leaf + Merkle proof, buildStealthTransferStatement, then TransactionBuilder -> resolve -> sign -> seal — the htlcClaim pipeline minus submit.	Also canonicalizeRelayedConditions(): rebuilds the relayed condition tree extracting BOTH leaf keys verbatim from the relayed tree — never substituting the account's own — so the security property of the relayed terms is preserved (see §4).
src/popup/*	Approval rendering for both kinds.	x402SignClaim copy says explicitly: “Sign only — not submitted to the network. The settlement service submits it later.” Shows what is being bought, price, fee, and refund deadline in wall-clock time.
src/background/capabilities.ts	signOnly: isLocal with an honest comment (verified live; does not claim delivery proof or facilitator honesty).	Flag stayed false-equivalent-honest until Phase 5 e2e passed, then caveats were updated — same pattern as scriptPathSpend.
docs/x402-facilitator.md (new)	The settlement interface spec: POST /x402/settle (idempotent on localDigest), GET /x402/settlement/{digest}, hold->broadcast->confirmed/expired state machine, nine-cause error contract, validation checklist, confidentiality rule (no preflight that sees the preimage), and a trust/honesty-semantics section.	Written so ANY ootle-indexer deployment can opt in; enabling settlement is a deployment decision, not a fork of the indexer.
docs/integration/x402.md (new)	dApp-facing guide: the 402 quote envelope, the flow, and a field-by-field alignment table against the x402 foundation repo's v2 specification.	Documents the four deliberate wire deviations from foundation v2 (flat envelope, scheme fields as extra, no CAIP-2 field, plain-JSON header with no PAYMENT-SIGNATURE analogue).
packages/x402-ootle (new)	@potr3288/x402-ootle: zero-dependency client package (pay402 fetch wrapper, pay(), settle(), refund on expiry), 34 tests.	Persisted outputMask is load-bearing: the funder cannot recover the refund path without it.
e2e/ (new, harness-only)	Playwright harness driving the real extension (real popup approvals, deterministic seed-derived wallets) + a local facilitator stand-in implementing the spec (24 tests).	Harness-only footprint: no production dependency added beyond the playwright devDep; src/ and packages/ otherwise frozen.

3. What was deliberately NOT changed

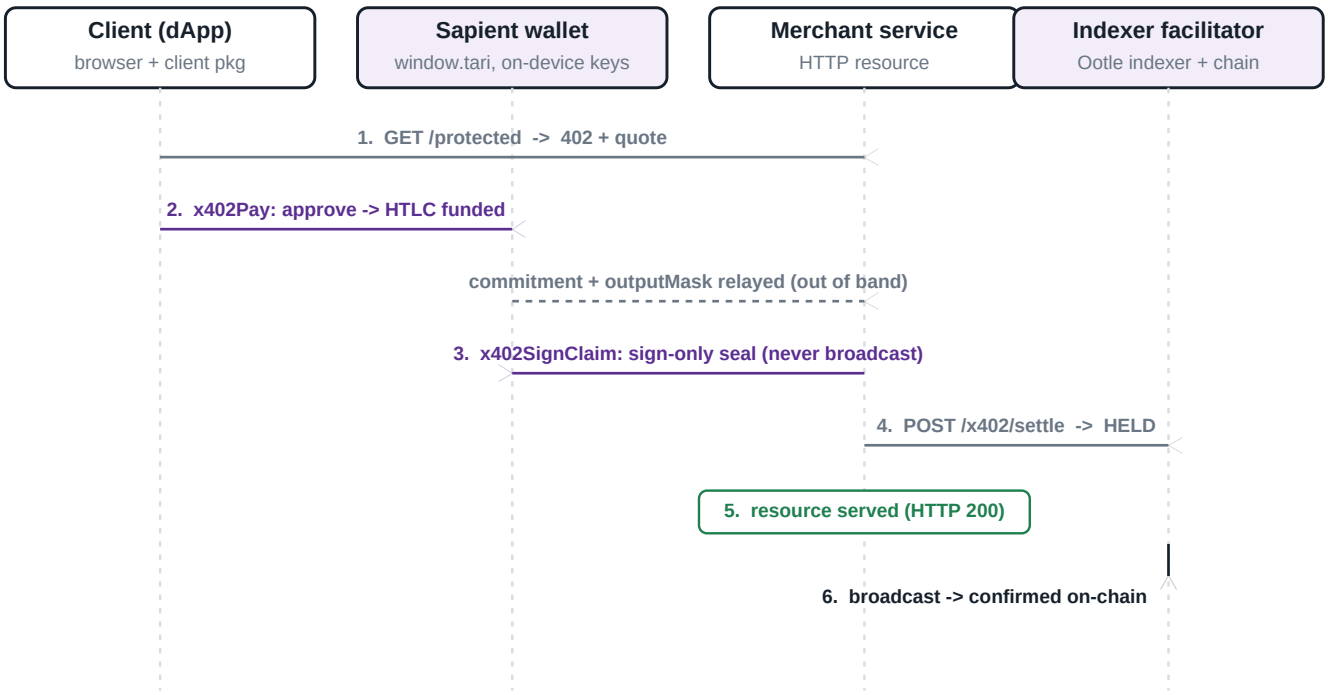
- No changes to the stealth/HTLC core, transaction building, or account model — the fork reuses htlcFund's execution path for x402Pay rather than duplicating builder logic.
- No new runtime dependencies (playwright is a devDependency of the e2e harness only).
- No dry-run/preflight anywhere in the settlement path: the SDK's claim path would otherwise POST the fully-signed transaction — preimage included — to the facilitator before settle. The preimage is the merchant's proof of payment; the facilitator spec forbids seeing it early, and the wallet's build path never dry-runs.
- Capability flags report honestly and stay tied to account kind (isLocal), matching the existing scriptPathSpend pattern.

4. SDK findings you'll want upstream

Four things surfaced during live verification that are relevant to @chironbuilder/ootle-sdk itself:

Finding	Detail	Suggested action
verifyHtlcTerms is key-order sensitive	It compares the rebuilt vs relayed HTLC condition tree with a raw JSON.stringify — byte-identical trees are rejected when key insertion order differs (extension storage canonicalizes key order; the SDK rebuild does not). Reproduced on 0.1.30.	Compare canonicalized JSON (sort keys recursively) or compare parsed structures. The fork works around it by rebuilding the relayed tree via htlcConditions before verification.
htlcClaim is submit-wired end to end	Every SDK claim path ends in submit; sign-only settlement flows need the pipeline minus submit. The low-level surface is sufficient today (buildScriptPathWitness + buildStealthTransferStatement + TransactionBuilder -> resolve -> sign -> seal all work), but it is non-obvious and undocumented.	Consider an official sign-only/seal-only variant of htlcClaim returning the sealed payload.
AfterEpoch(N) is strict	Live-verified: AfterEpoch(12754) rejected while executing at epoch 12753, satisfied from 12759 = refundEpoch 12758 + 1 (run V). Spendable at N+1, not N.	Worth documenting in the HTLC condition docs; it is easy to misread and cost a test run.
sendPrivately outputs are not view-key scannable	A recipient cannot discover the transfer by scanning with the view key; recipientCommitment must be delivered out of band (the payer's result carries transactionId, recipientCommitment, recipientSubstateId, minimumValuePromise).	Document the out-of-band requirement; the claim popup path (popup-claim-private-payment) is the discovery mechanism for a live wallet.

5. Verification summary



Check	Result
Unit tests (extension)	278/278 green, including per-rejection-path validation tests and real-wasm statement round-trips.
Client package	34/34 green.
Facilitator stand-in vs spec	24/24 green — one test per error code, idempotency, expired terminal state.
Typecheck / build	Clean at aee37c8.
Live esmeralda happy path	PASS — claim tx 21257dd3cb0e… finalized Commit, HTLC output spent, resource served after hold.
Live esmeralda refund	PASS — facilitator frozen (claim held forever, never broadcast); settlement reported expired; funder htlcRefund tx 52a51a16ee86… finalized Commit, HTLC output refunded at epoch 12759.

The refund run is the interesting one for a maintainer: it exercises the exact failure mode the design exists for — a facilitator that takes the signed claim and never broadcasts — and shows the chain, not the facilitator, returning the funds.

6. Reviewing the fork

Branch x402/quote-and-signonly on github.com/potr3288/tari-wallet, base e2f7bf9. Key commits: b251547/8828a7a (envelope + honest capability), 6f24c8c (e2e harness), e82cf1c (SDK key-order workaround + facilitator dry-run removal), b2e2321 (refund harness fixes), 1b96487 (honesty semantics), aee37c8 (dApp doc + foundation cross-check). The diff to read first: x402SignOnly.ts, then the dispatch/validation cases in index.ts, then the facilitator spec. Everything upstream-relevant beyond the wallet itself lives in docs/x402-facilitator.md — the indexer-side implementation is pinned by that interface, not by this fork.